

P395: Guide 6: Linear Algebra

1. Start up a new notebook and title it 'Guide6_your_name'. Enter

```
import matplotlib.pyplot as plt
import numpy as np
```

Linear Algebra: Intro

This week we are going to look at solving various problems from linear algebra. This usually involves solving a system of linear equations for some unknown variables which can be conveniently summarized in matrix form as $Ax = b$, where each row of the matrix A corresponds to one of the equations and contains the coefficients, x is the vector of unknown variables that we are trying to solve for, and b is a vector of constants. Many problems in physics can be cast in this form, and linear algebra deals with determining whether there exists a solution and, if so, finding it. Python has many fast routines for solving linear-algebra problems, and we will use a few of them in this week's lab.

Task 1: Vector Operations

Before we dive into solving systems of equations, we will look at various vector and matrix operations and how they are implemented in python. It is important to always be working with NumPy arrays (*i.e.*, `np.array( ... )`) in order to do these operations, as python lists will not work (as you saw in Guide 0).

Addition/subtraction:

Addition and subtraction of vectors or arrays in Python is straightforward, but again, they must be NumPy arrays. Let's do a quick reminder of mathematical operations on NumPy arrays.

1. You have two vectors $a = (1.1, -2.1, 0.0)$ and $b = (2.0, 1.1, -0.5)$. Try adding them as Python lists using `[1.1, -2.1, 0.0] + [2.0, 1.1, -0.5]`. What do you get? Then try `np.array([1.1, -2.1, 0.0]) + np.array([2.0, 1.1, -0.5])`. Which one gave the correct answer for $a + b$?

Multiplication by a scalar

2. You want to calculate $2.3a$. Try calculating it using a Python list for a and then with a as a NumPy array. Which one worked?

You can do the same multiplication by a scalar for multi-dimensional arrays as well.

Vector products

All the various vector products are also available in python. But note, you can also readily multiply two arrays together and the result might not be the product you intended.

3. Take your NumPy arrays a and b from above and calculate $a*b$. What is the result?

Dot product

The vector dot product is $a \cdot b = \sum_i a_i b_i$. This is implemented with the NumPy command `np.dot`.

4. Calculate the dot product of a and b .
5. Calculate `np.sum(a*b)` and check that it agrees with what you found in (4).

Cross product

The cross product of two vectors is a new vector that is orthogonal to the other two. It can be defined

in terms of a determinant as $a \times b = \begin{vmatrix} \hat{i} & \hat{j} & \hat{k} \\ a_x & a_y & a_z \\ b_x & b_y & b_z \end{vmatrix}$.

6. The command `np.cross` computes the cross product. Calculate $a \times b$.
7. Analysis: Work out the determinant by hand using your values for a and b . Does your answer agree with what you found in (6) ?

Vector norm

Given a vector, you may want to calculate its length (or norm), $|a| = \sqrt{a \cdot a}$.

You can calculate the norm of a vector using `norm` from the `scipy.linalg` package.

8. Import the package by executing `import scipy.linalg as la`
9. Calculate the norm of the vector a above using `la.norm` as well as the above definition in terms of the dot product. They should agree.

Task 2: Matrices and 2D arrays

Matrix product

For matrices, the dot product becomes the matrix product. If you have matrices (*i.e.*, 2D arrays) M and P , their matrix product $Q = MP$ is $Q_{i,j} = \sum_k M_{i,k} P_{k,j}$. Note that M^*P is not the matrix product of M and P . It is just the (element-wise) product of all the elements in each array.

1. Define the arrays $\sigma_x = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ and $\sigma_z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$.
2. Calculate the matrix product $\sigma_x \sigma_z$ using `np.dot`.
3. Calculate $\sigma_x * \sigma_z$ and confirm that it is indeed not the matrix product that you found in (2).
4. Calculate the matrix product of $\sigma_z \sigma_x \sigma_z$.

Array constructors

NumPy also has a variety of array constructors for making arrays quickly. All these commands return NumPy arrays. The command `np.zeros` returns an array (of specified dimension) that is all zeros.

The command `np.ones` returns an array (of specified dimension) that is all filled with 1's. And the command `np.identity` returns an identity matrix of a given size. These are the most commonly used constructors, but if there is an exotic matrix that you need to build, there might be a NumPy command that will generate it for you.

5. Make a 3x4 array of zeros and print it out.
6. Make the 5x5 identity matrix and print it out.

Matrix operations

Many commonly used matrix operations are also available in NumPy. To take a transpose of a matrix A , simply do `A.T`. And to compute an inverse of a matrix A , you can use the command `la.inv(A)` from the linear algebra package.

7. Make a random 3x3 matrix using `np.random.random` and print it out.
8. Take the transpose of the matrix in (7) and print it out.
9. Calculate the inverse of the matrix in (7) and print it out.
10. Calculate the matrix product of the inverse found in (9) with the matrix from (7) and confirm that it is indeed the identity matrix.

Background: Solving linear systems of equations

Many problems in physics involve solving a system of linear equations. Here we consider solving a system of 3 linear equations in 3 unknowns (x_1, x_2, x_3) ,

$$\begin{aligned}a_{11}x_1 + a_{12}x_2 + a_{13}x_3 &= b_1 \\a_{21}x_1 + a_{22}x_2 + a_{23}x_3 &= b_2 \\a_{31}x_1 + a_{32}x_2 + a_{33}x_3 &= b_3\end{aligned}$$

which can be summarized in matrix form as $Ax = b$, where the matrix A is

$$A = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix}, \text{ the vector } x \text{ is } x = (x_1, x_2, x_3), \text{ and the vector } b \text{ is } b = (b_1, b_2, b_3). \text{ If one}$$

could find the inverse A^{-1} of the matrix A , then one could find the solution as $x = A^{-1}b$; however, finding the inverse can be numerically difficult, leading to overflow or underflow if A is nearly singular. In your introductory linear algebra course you would have learned the Gauss-Jordan method for manipulating the matrix A and solution vector b to row reduce A to a point where the matrix is upper-triangular, giving

$$\begin{pmatrix} a'_{11} & a'_{12} & a'_{13} \\ 0 & a'_{22} & a'_{23} \\ 0 & 0 & a'_{33} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} b'_1 \\ b'_2 \\ b'_3 \end{pmatrix}$$

This is then easily solved by back substitution (*i.e.*, $x_3 = b'_3/a'_{33}$, which then gets substituted into the previous equation that can then be solved for x_2 , and so on). One could also do the reverse and reduce the equations so that the matrix is lower triangular and then do forward substitution. However, Gauss-

Jordan involves many operations ($\sim N^3$, for N equations) and can be done more efficiently with different algorithms. Here we detail the one used by python.

Task 3: LU decomposition

If we can get the above matrix A into triangular form, then the equations become straightforward to solve. An efficient method for doing this, faster than Gauss-Jordan elimination, is called LU decomposition. It decomposes A into two triangular matrices, a lower triangular matrix L and an upper triangular matrix U such that $A = LU$. In the context of solving a linear system $Ax = b$, this becomes $L(Ux) = b$. If we set $Ux = y$, then we just need to solve $Ly = b$ first and then $Ux = y$ second to get our solution x . This is how Python solves a linear system.

To solve linear systems in python use `la.solve`. We'll now use it to solve a 5D system of linear equations.

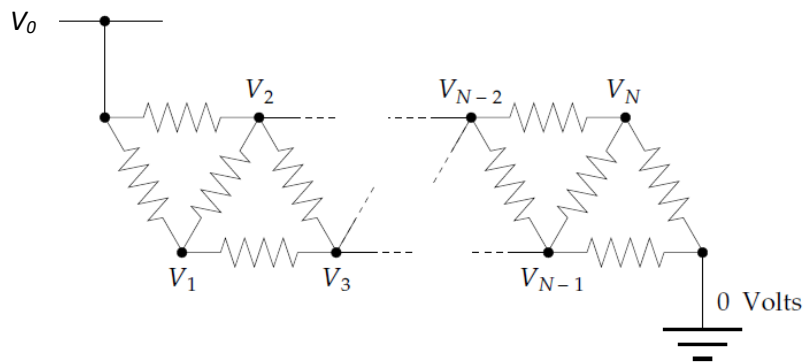
1. Make a random 5 x 5 matrix A and a random vector b of length 5. Print them out.
2. Use `la.solve` to find the solution to $Ax = b$.
3. Calculate the dot product of A with the solution x and confirm that it equals b .

`la.solve` carried out LU decomposition on A to solve the system of linear equations. Let's look at the steps that `la.solve` actually carries out behind the scenes:

4. Perform LU decomposition by using `la.lu` on the matrix A . (Note: In doing LU decomposition, the algorithm may have to permute some of the rows of A . By looking up the help on `la.lu`, you'll see that it returns P, L, U where P is the permutation matrix such that $PLU = A$.) Print out P, L , and U .
5. Calculate the matrix product PLU and confirm that it is the same as A .
6. Now using L and U , i) calculate the solution to $Ly = b' = P^{-1}b$ for y , and then ii) calculate the solution to $Ux = y$ for x .
7. Analysis: Is your result for x in (6) the same as you found in (2)? It should be.

Task 4: Application of solving linear equations: Resistor chain circuit

Consider the following resistor chain circuit. There is an applied voltage $V_0=4$ volts on one side, with the other side grounded at zero voltage. All the resistors have the same resistance R . The goal of this problem is to find what the voltages are at all the interior points of the circuit.



1. Analysis: Use Ohm's law ($I = \Delta V/R$) and Kirchhoff's current law (*i.e.*, the sum of the current entering/leaving a node must be zero) to show that the voltages satisfy the following equations,

$$\begin{aligned}
 3V_1 - V_2 - V_3 &= V_0 \\
 -V_1 + 4V_2 - V_3 - V_4 &= V_0 \\
 &\vdots \\
 -V_{i-2} - V_{i-1} + 4V_i - V_{i+1} - V_{i+2} &= 0 \\
 &\vdots \\
 -V_{N-3} - V_{N-2} + 4V_{N-1} - V_N &= 0 \\
 -V_{N-2} - V_{N-1} + 3V_N &= 0
 \end{aligned}$$

2. Analysis: Express the above equations in the form $Ax = b$, where $x = (V_1, V_2, \dots, V_N)$, to find the matrix A and vector b .
3. Use `la.solve` to solve $Ax = b$ for the unknown voltages V_i for a chain where $N=6$ and the external voltage $V_0=4$ volts. (NOTE: Your solution should have all voltages < 4 volts. If not, then there's a problem with your equations.) Print out your solution.

Now consider the case where $N=10,000$. You should notice that most of A 's entries are zeros (*i.e.*, there's only a narrow band of non-zero elements around the diagonal). Storing such a large matrix could cause memory problems. Fortunately, there is still a way to solve this system.

4. Look up the help on `la.solve_banded` to solve the $N=10,000$ case.
5. From the `la.solve_banded` documentation, figure out how to pass your matrix A in banded form (which takes up a lot less space).
6. Use `la.solve_banded` to solve for the unknown V 's.
7. Plot the solution V_i . How does the voltage drop across the network of resistors?

Task 5: Eigensystems

Another common matrix problem that one must solve in physics is to find the eigenvalues λ and eigenvectors x that satisfy the equation

$$Ax = \lambda x.$$

You should have seen in your linear algebra course that to solve this system we solve $\det(A - \lambda I) = 0$. This leads to a characteristic polynomial in λ , the roots of which are the eigenvalues. The eigenvalues can then get substituted back into the original equations to solve for the eigenvectors x .

In practice, one never uses this approach to numerically find the eigenvalues and eigenvectors. Instead, one seeks a set of matrix transformations that when applied to the matrix A make it diagonal. The types of transformations are dependent on the nature of A .

Which of Python's eigensystems algorithms to use is determined by the type of matrix to diagonalize. The python command `la.eig` is a general-purpose eigenvalue/eigenvector solver that will work on any matrix. If your matrix is real and symmetric (or Hermitian), you can use the slightly faster `la.eigh`. If you just want the eigenvalues, you can call `la.eigvals` (or `la.eighvals`).

1. Generate a random 8 x 8 matrix that we'll use to calculate the eigenvalues.

First we'll find the eigenvalues by solving the characteristic equation. The command `np.poly` returns the characteristic equation (as coefficients of each term in the polynomial) of a matrix A .

2. Use `np.poly` to calculate the polynomial coefficients of your matrix.
3. The command `np.roots` calculates the roots of a polynomial. Pass your characteristic polynomial coefficients found in (2) to `np.roots` to find the eigenvalues of your random matrix.

Now we'll use python's eigenvalues solver to do the same thing

4. Use `la.eigvals` to find the eigenvalues of your matrix. They should agree with what you found in (3).

Task 6: Eigenvalue Problem Application

Normal modes of a protein: Gaussian Network Model

A physics problem that requires the calculation of eigenvalues and eigenvectors is finding the normal modes of an elastic structure. Consider an object that is made up of point masses (nodes) connected by springs. The nodes will have an equilibrium set of positions where the net force of the springs is zero. But there will also be motions that bend and distort the nodes. For such an object, one can find the normal modes which correspond to orthogonal motions of the springs, sorted by their frequencies. Soft modes correspond to collective motions of the nodes that have low frequency. Using Newton's 2nd law for the motion of each node and assuming a sinusoidal response leads to an eigenvalue/eigenvector problem for the motions of the nodes (see Wikipedia for an OK description).

A nice application of normal-mode analysis is to the study of the motions of proteins. Proteins are biomolecules typically made up of 1,000s of atoms. The collective motion (*i.e.*, soft modes) of these atoms are often important for the protein's function. A model called the Gaussian network model decomposes the motions of the atoms in a protein into its normal modes. The normal modes are found by calculating the eigenvalues/eigenvectors of the Kirchhoff matrix Γ (you'll have already seen a similar matrix in the resistor chain problem). The off-diagonal elements of this matrix are defined based on the pairwise distances between the atoms in the protein as

$$\Gamma_{i,j} = -1 \quad \text{if } r_{i,j} < r_c$$

$$\Gamma_{i,j} = 0 \quad \text{otherwise}$$

where $r_{i,j}$ is the pairwise distance between the pair of atoms (i,j), and r_c is a specified interaction distance cut-off ($r_c = 7$ Angstrom is reasonable for proteins). The diagonal elements of Γ are then set to be

$$\Gamma_{i,i} = - \sum_{j \neq i} \Gamma_{i,j}$$

The eigenvalues λ_i of the spring matrix give the frequencies of the modes, and the eigenvectors $\vec{u}^i$ give the size of the motion at each position. There is always a zero-frequency mode that corresponds to pure translation (*i.e.*, a constant eigenvector). The remaining non-zero frequencies are the vibrational normal modes. The low-frequency modes are the collective motions.

We will now do some normal-mode analysis on the protein that carries out insertion of the HIV virus into the host genome (HIV reverse transcriptase). You can read a nice summary of this protein at <https://pdb101.rcsb.org/motm/33>.

1. Switch to the python `%matplotlib widget` environment, as we will do some interactive 3D plots.
2. Download the file '1rev_CAs.txt' from Canvas. This file contains the positions (x, y, z), in units of Angstroms, of the main backbone carbon atoms of the HIV reverse transcriptase protein
3. Read in the x,y,z positions from the file into an array.
4. How many backbone carbon atoms are in the protein?
5. Make an interactive 3D scatter plot of the positions. Make your points big enough to 'space fill' the volume of the protein. You should be able to rotate and see the 'claw' of this protein which grabs and binds DNA and RNA.
6. Calculate a matrix of all the pairwise distances between each atom.
7. Using this matrix and a cutoff distance of $r_c = 7$ Angstrom, write code to construct the Kirchhoff matrix for this protein.
8. Calculate the eigenvalues/eigenvectors of the Kirchhoff matrix.
9. Print out the first 5 eigenvalues. The first one should be ~ 0 . The rest should be positive and increasing in size. If not (*i.e.*, if you have negative values), then you haven't calculated the correct Kirchhoff matrix.
10. Make another 3D scatter plot of the protein, but now color the points using the squared amplitudes of the lowest non-zero frequency eigenvector.
11. Analysis: Where are the dominant collective motions of this mode (*i.e.*, the red/hot spots) located on this protein in relation to the 'claw'?
12. Make another 3D scatter plot, but now using the 2nd-lowest nonzero frequency eigenvector. (You could keep going, looking at increasingly higher-frequency modes, but we'll stop at the 2nd).
13. Analysis: How was normal-mode analysis helpful in visualizing the potential collective motions of this protein?

Task 7: Wrap up

1. Please provide any feedback on questions that you found difficult to understand because of the wording.
2. Please provide any other comments about this notebook.